LIVE VERIFICATION PROCEDURES

M.O.A.C. v3.1 — New AGI Capabilities

June 13, 2026 · Defensive Publication · Public Distribution

Author: Melvin E. Bellinger Jr. — Connecticut, United States

LICENSE AND PATENT RIGHTS

PURPOSE

This document provides reproducible procedures for independently verifying each new AGI capability added in M.O.A.C. v3.1. Each procedure issues a specific request against the running system and states the expected observable result. Where a procedure does not produce the stated result, that item should be disregarded and is not claimed as verified.

Prerequisites:

- System running at `http://localhost:3002`
- All four new modules confirmed loaded (`GET /api/v3/agi/state` returns `"success": true`)
- Any HTTP client: `curl` , a web browser, Postman, or equivalent

---

VERIFICATION 1: System Observability

Capability tested: Self-preservation, observable cognitive state

Request:

```
GET http://localhost:3002/api/v3/agi/state
```

Expected result: HTTP 200 with a JSON body containing `"success": true` and four top-level subsystem keys under `"agi"`: `episodicMemory`, `knowledgeGraph`, `curiosity`, `systemHealth`. Under normal operating conditions, `systemHealth.ok` should be `true` and `systemHealth.warnings` should be an empty array.

What this demonstrates: The system exposes a machine-readable internal state endpoint returning live snapshots of four independently maintained, persistent data structures. This is not a static status page — each field reflects the current runtime state of a distinct cognitive subsystem.

VERIFICATION 2: Metacognition — Confidence Boundary Detection

Capability tested: Metacognition, explainability

Procedure: Send a query whose domain is classified as uncertain (real-time data, personal predictions, future events). Observe whether the response explicitly acknowledges the limitation.

Uncertain-domain query:

```
POST http://localhost:3002/api/v3/query/stream
Content-Type: application/json

{
  "query": "What will the stock market do tomorrow?",
  "sessionId": "verify-metacog"
}
```

Expected result: The response contains explicit hedging — e.g., "*I can't predict that with any reliability*" or "*there is no way to know with certainty*" — rather than a confident fabricated forecast. The AGI pipeline will have classified this query's confidence below 0.50 (future prediction domain) and injected a knowledge-boundary directive before the LLM call.

High-confidence control query:

```
{
  "query": "What is a for loop in Python?",
  "sessionId": "verify-metacog"
}
```

Expected result: A direct, confident explanation with no hedging language. The pipeline will have scored this query at ≥ 0.75 (programming domain) and injected no boundary directive.

What this demonstrates: The system modulates its expressed confidence based on a live per-query domain assessment — not a blanket disclaimer applied to all responses.

VERIFICATION 3: Causal Reasoning — Counterfactual and Chain Detection

Capability tested: Causal reasoning

Counterfactual query:

```
POST http://localhost:3002/api/v3/query/stream
Content-Type: application/json

{
  "query": "What would have happened if the internet had never been invented?",
  "sessionId": "verify-causal"
}
```

Expected result: The response explicitly reasons about the alternate timeline — describing what the world would look like without the internet — rather than deflecting with "*it's impossible to say.*" The AGI pipeline detects the `counterfactual` pattern from the phrase "*what would have happened if*" and injects the directive: "*This is a counterfactual — reason about the alternate timeline explicitly.*"

Causal chain query:

```
{
  "query": "Why does deforestation cause flooding?",
  "sessionId": "verify-causal"
}
```

Expected result: The response traces an explicit cause-to-effect chain (tree root systems → water absorption → surface runoff → flooding) rather than providing a one-sentence answer. The pipeline detects the `why-explanation` causal pattern and injects a trace-the-chain directive.

What this demonstrates: Causal query type is detected per-turn and shapes the reasoning directive given to the model before it generates a response.

VERIFICATION 4: Theory of Mind — Emotion and Expertise Detection

Capability tested: Theory of Mind, social intelligence

Frustrated-user query:

```
POST http://localhost:3002/api/v3/query/stream
Content-Type: application/json

{
  "query": "This code keeps breaking and I don't understand why it won't work",
  "sessionId": "verify-tom"
}
```

Expected result: The response acknowledges the frustration before offering solutions — e.g., "*That's a frustrating situation — let's work through what's going wrong.*" The pipeline detects the `frustrated` emotional state from signal phrases "*keeps breaking*" and "*won't work*" and injects: "*User seems frustrated — acknowledge the problem before solving.*"

Beginner-level query:

```
{
  "query": "What is a variable?",
  "sessionId": "verify-tom-beginner"
}
```

Expected result: A plain-language explanation with no assumed technical background. The pipeline detects a `beginner` expertise signal from the "*What is*" phrasing pattern.

Expert-level query:

```
{
  "query": "How do I implement a lock-free concurrent queue using CAS operations?",
  "sessionId": "verify-tom-expert"
}
```

Expected result: A technical response without basic definitions or preamble. The pipeline detects `expert` signals from "*lock-free*", "*concurrent*", and "*CAS operations*".

What this demonstrates: The system dynamically adapts communication style based on per-turn inference of the user's emotional state and expertise level.

VERIFICATION 5: Episodic Memory — Cross-Turn Recall

Capability tested: Long-term episodic memory, persistent recall

Step 1 — Establish an episode:

```
POST http://localhost:3002/api/v3/query/stream
Content-Type: application/json

{
  "query": "Tell me about the Doppler effect",
  "sessionId": "verify-episodic"
}
```

Allow the response to complete fully before proceeding. **Step 2 — Trigger episodic recall (separate session ID):**

```
{
  "query": "How does wave frequency relate to motion?",
  "sessionId": "verify-episodic-2"
}
```

Expected result: After several exchanges, the `EPISODIC MEMORY` block in the volatile context will include the prior Doppler exchange as a relevant past episode. This can be verified by polling the state endpoint.

State confirmation:

```
GET http://localhost:3002/api/v3/agi/state
```

`episodicMemory.episodeCount` should increment by one after each completed exchange, confirming storage.

What this demonstrates: The system retains past exchanges across sessions using file-backed storage and retrieves them by TF-IDF semantic similarity — not keyword match or session scope alone.

VERIFICATION 6: Knowledge Graph — Concept Accumulation and Transfer Learning

Capability tested: Continual learning, transfer learning, open-world concept acquisition

Step 1 — Trigger any exchange.

Step 2 — Check graph state:

```
GET http://localhost:3002/api/v3/agi/state
```

Expected result: After any exchange, `knowledgeGraph.nodeCount` > 0 and `knowledgeGraph.edgeCount` > 0. The `topConcepts` array reflects the most frequently encountered concepts.

Transfer learning check: After exchanges spanning at least two domains (e.g., one programming query and one biology query), a concept that appeared in both will carry multiple domain labels. This can be verified by inspecting `data/knowledge-graph.json` — any node with a `domains` field containing two or more entries is a transfer-learning candidate.

Growing graph test: Submit five distinct queries across different topics. After each, `knowledgeGraph.nodeCount` and `knowledgeGraph.turnCount` should both increment, confirming continual growth.

What this demonstrates: The knowledge graph grows on every conversation turn without explicit training, retraining, or fine-tuning. Concepts that appear in multiple domains accumulate cross-domain labels, implementing the operational mechanism of transfer learning.

VERIFICATION 7: Intrinsic Motivation — Knowledge Gap Detection and Goal Generation

Capability tested: Intrinsic motivation, self-directed curiosity, autonomous goal generation

Step 1 — Send a query the system cannot fully answer:

```
POST http://localhost:3002/api/v3/query/stream
Content-Type: application/json

{
  "query": "What is the exact population of every city in the world right now?",
  "sessionId": "verify-curiosity"
}
```

Expected result: The response will hedge (real-time data beyond knowledge cutoff). The curiosity engine scans the response for hedge language, records the topic as a knowledge gap, and automatically generates an exploration goal for it.

Step 2 — Check gap and goal state:

```
GET http://localhost:3002/api/v3/agi/state
```

After several hedged responses, `curiosity.knowledgeGaps` will contain recorded gap entries and `curiosity.activeGoals` will contain auto-generated exploration goals derived from those gaps.

Curiosity pulse check: After 5 or more exchanges across different topics, `curiosity.curiosityPulse` will return the top 3 topics the system is currently most curious about. This list is dynamic — a topic discussed repeatedly will decay in priority; a topic not recently discussed will rise.

What this demonstrates: The system autonomously detects the boundaries of its own knowledge, records them, and generates self-directed exploration goals without any external reward signal or human instruction to do so.

VERIFICATION 8: Semantic Fact Extraction — Lifelong Learning

Capability tested: Semantic memory, lifelong fact accumulation

Step 1 — Send a factual query:

```
{
  "query": "What is photosynthesis?",
  "sessionId": "verify-semantic"
}
```

Step 2 — Check semantic memory (allow a brief pause for background processing):

```
GET http://localhost:3002/api/v3/agi/state
```

Expected result: `episodicMemory.semanticFactCount` > 0. The system has extracted declarative sentences from the response (e.g., "*Photosynthesis is the process by which...*") and stored them as semantic facts with associated confidence scores.

Confidence increment test: Ask the same factual topic a second time. The semantic fact's confidence score will increment by 0.05 on re-observation (deduplication via cosine similarity), up to a maximum of 1.0.

What this demonstrates: The system extracts and stores structured knowledge from its own responses at runtime, building a persistent semantic fact base that grows and gains confidence with repeated observation — without any separate training or fine-tuning step.

VERIFICATION 9: Self-Preservation — Health Monitor

Capability tested: Self-preservation, resource-aware operation

Request:

```
GET http://localhost:3002/api/v3/agi/state
```

Expected fields:

| Field | Normal State | Threat State | |---|---|---| | `systemHealth.ok` | `true` | `false` | | `systemHealth.warnings` | `[]` | Array of warning strings |

Disk threat simulation (optional): Fill the disk to near-capacity. On the next health check cycle (up to 30 seconds), `warnings` will include `"DISK_WRITE_FAILED"`. The system continues operating; the warning is surfaced in the AI's response context via the `[AGI-HEALTH]` directive.

Memory pressure simulation (optional): Under heavy concurrent load, if heap usage exceeds 3.5 GB, `warnings` will include a `"MEMORY_HIGH"` entry with the current heap size. The AI will be aware of its own memory pressure before generating a response.

What this demonstrates: The system actively monitors its own operational resources and informs the model of resource constraints before each response — a property distinct from passive crash handling or external process monitoring.

VERIFICATION SUMMARY

| # | Capability Verified | Endpoint / Method | Key Observable | |---|---|---|---| | 1 | System observability | `GET /api/v3/agi/state` | 4 subsystem keys present; `success: true` | | 2 | Metacognition | `POST /api/v3/query/stream` | Confident on known domains; hedges on uncertain | | 3 | Causal reasoning | `POST /api/v3/query/stream` | Counterfactual reasoning + explicit causal chains | | 4 | Theory of Mind | `POST /api/v3/query/stream` | Emotional acknowledgment; expertise-matched depth | | 5 | Episodic memory | `GET /api/v3/agi/state` | `episodeCount` increments each turn | | 6 | Knowledge graph / transfer learning | `GET /api/v3/agi/state` | `nodeCount` + `edgeCount` grow each turn | | 7 | Intrinsic motivation | `GET /api/v3/agi/state` | `knowledgeGaps` + `activeGoals` populated after hedged responses | | 8 | Semantic learning | `GET /api/v3/agi/state` | `semanticFactCount` grows after factual exchanges | | 9 | Self-preservation | `GET /api/v3/agi/state` | `systemHealth.ok` reflects live disk + memory state |

Defensive publication. Intended for public distribution. Procedures describe the author's own running system as observed on June 13, 2026.

— Melvin E. Bellinger Jr., June 13, 2026